Project Proposal & Scope Document

DeployGuard:

A Reliable Repository Scanning and Deployment Orchestration System

Submitted By

Hassan Sajjad - 232460

Faria Fatima - 233169

Tania Khawar - 232378

Supervisor

Sir Yaseen Mushtaq

Co-Supervisor

Sir Asim Ali Fayyaz



Department of Computer Science
Air University Islamabad (AU)

Supervisor Meeting Log

Date	Supervisor	Discussion Points	Supervisor Signature
05-02-2026	Sir Yaseen Mushtaq	Initial project concept for "DeployGuard" presented. Discussion focused on solving the DevOps skill gap for SMEs via a "zero-configuration" pipeline. Feasibility of mapping code to infrastructure automatically and using AI for troubleshooting was reviewed.	
11-02-2026	Sir Yaseen Mushtaq	Technology stack finalized: Next.js for frontend, NestJS/Node.js for the orchestration engine, and PostgreSQL for audit logging. Supervisor approved the use of HashiCorp Terraform for IaC and Redis/BullMQ for managing asynchronous deployment queues.	
18-02-2026	Sir Yaseen Mushtaq	Detailed review of the Rule-Based Stack Detection engine to eliminate AI hallucination risks. finalized "Shift-Left" security requirements using Aqua Trivy and pre-deployment financial transparency via Infracost API integration.	
26-02-2026	Sir Yaseen Mushtaq	Final module list confirmed, including AWS S3/DynamoDB state locking, AI-powered log analysis, and the "No Lock-in" Terraform export feature. Project proposal signed.	

Project Proposal & Scope Document 1
DeployGuard: A Reliable Repository Scanning and Deployment Orchestration System 1
Supervisor Meeting Log 2
1. Introduction 4
2. Related System Analysis / Literature Review 4
3. Problem Statement 4
4. Problem Solution / Objectives of the Proposed System 5
5. Scope 5
6. Modules 5
7. System Limitations/Constraints 8
8. Data Gathering Approach 8
9. Tools and Technologies 9
10. Project Stakeholders and Roles 9
11. Module based Work Division 9
12. Gantt Chart 11
13. Mockups 11
14. References 13

1. Introduction

The software development sector is growing rapidly, yet startups and Small-to-Medium Enterprises (SMEs) remain heavily constrained by the complexities of cloud deployment. Developers often struggle to configure reliable infrastructure, while businesses are deprived of the cost savings associated with optimized cloud primitives due to a severe DevOps skill gap. Additionally, traditional manual deployments are highly prone to human error, resulting in security vulnerabilities, unstandardized environments, and slow-release cycles.

Recognizing these challenges, we propose the development of "DeployGuard," an enterprise-ready, cloud-native DevOps orchestration platform. This platform transforms the deployment experience by introducing a "zero-configuration" pipeline that maps code to infrastructure automatically via safe, rule-based scanning. Furthermore, an integrated Conversational AI Assistant actively troubleshoots deployment errors, bridging the technical gap for developers. DeployGuard provides a strictly governed, secure pipeline that enhances deployment efficiency and operational confidence.

2. Related System Analysis / Literature Review

Application Name	**Weakness**	**Proposed Project Solution (DeployGuard)**
Raw AWS Console	Extremely steep learning curve; requires deep DevOps knowledge to configure VPCs, ALBs, and ECS manually.	Abstracts the complexity of AWS, using an automated orchestration engine to deploy code seamlessly without requiring raw console access.
Standard PaaS (e.g., Heroku)	Heavily abstracted but very expensive at scale. Lacks transparent cost estimation before deploying.	Deploys directly to the user's AWS account using Fargate Spot instances for massive cost savings, integrating Infracost for pre-deployment financial transparency.
Basic GitHub Actions	Only handles CI/CD; lacks built-in infrastructure provisioning (IaC) state locking and mandatory security gates.	Enforces mandatory Aqua Trivy security scanning and manages HashiCorp Terraform state locking via S3/DynamoDB to prevent concurrent deployment collisions.

3. Problem Statement

The absence of accessible, secure, and cost-effective cloud orchestration for SMEs represents a significant bottleneck in software delivery. Existing solutions operate predominantly on two extremes: either heavily abstracted but expensive managed platforms, or raw cloud primitives that require dedicated, highly paid DevOps engineers. As a result, small teams rely on manual, unstandardized deployment scripts. Without integrated pre-deployment cost estimators or automated security gates, these teams frequently face unexpected cloud "bill shock" and inadvertently push vulnerable container images to production environments. Furthermore, when complex cloud deployments fail, developers waste hours parsing cryptic log files without clear remediation steps.

4. Problem Solution / Objectives of the Proposed System

To address the limitations of current workflows, DeployGuard introduces a robust orchestration platform. To completely eliminate the risks of AI code hallucination during containerization, the system utilizes strict rule-based repository scanning to map application stacks directly to highly optimized, pre-stored Docker templates. This automates architectural decisions while guaranteeing stability. Secure infrastructure provisioning via HashiCorp Terraform (with distributed state locking) ensures deployment reliability. Finally, to eliminate the friction of debugging, an integrated AI Assistant parses failed logs and provides plain-English solutions.

4.1 Objectives

- To develop a "zero-configuration" cloud deployment platform tailored for startups and SMEs.
- To eliminate manual configuration errors by utilizing rule-based repository scanning mapped to pre-stored DevOps Docker templates.
- To enforce mandatory "shift-left" security by integrating Aqua Trivy to block the deployment of critical vulnerabilities.
- To provide absolute financial transparency by integrating Infracost for pre-deployment AWS cost estimations.
- To prevent infrastructure state corruption during concurrent deployments using Amazon S3 and DynamoDB remote state locking.
- To integrate an AI-powered Troubleshooting Chatbot (LLM) that automatically analyzes failed deployment logs, pinpoints the exact point of failure, and provides actionable remediation steps.

5. Scope

The scope of this project is to develop a DevOps orchestration web application for seamless cloud deployments. Users will link their GitHub repositories, and the system will automatically scan, containerize, and deploy the application to AWS ECS Fargate Spot instances. The platform features robust CI/CD automation, a vulnerability reporting dashboard, pre-deployment cost analysis, and IaC provisioning using Terraform. The scope includes an integrated LLM chatbot strictly focused on platform guidance, log analysis, and error detection to assist users when builds or deployments fail.

6. Modules

6.1 User Authentication & Audit Logging

- Allow users to securely sign up and log in using GitHub OAuth and standard email/password credentials.
- Implement role-based access control (RBAC) to differentiate between Admin, Developer, and Read-Only users.
- **Audit Logging:** Immutably record every system action (deployments, Trivy failures, cost approvals) in a highly available PostgreSQL database via TypeORM for enterprise compliance.

6.2 Project & Workspace Management

- Allow users to create isolated "Projects" within their workspace and link specific GitHub repositories to them.
- Enable users to select target branches for deployment and securely manage environment variables (.env).

6.3 Rule-Based Stack Detection Engine

- Implement NestJS backend logic to perform shallow clones of the linked repositories to minimize network I/O.
- Scan the repository's Abstract Syntax Tree (AST) and file signatures (e.g., package.json, requirements.txt) to automatically identify the runtime environment.

6.4 DevOps Template Engine & Pre-Flight Validation

- Maintain a secure registry of optimized Docker templates utilizing distroless/alpine base images and multi-stage builds.
- **Pre-Flight Validation:** Generate a structured JSON report detailing the detected stack and injected template for user review before building.

6.5 CI/CD Pipeline & Asynchronous Queuing (Redis)

- **Asynchronous Execution:** Utilize Redis and BullMQ to queue long-running Terraform and Docker build jobs, ensuring the NestJS backend remains highly responsive.
- Automatically trigger GitHub Actions workflows, build the image, and immutably tag it with its unique Git commit hash.
- Push artifacts to AWS ECR and enforce **ECR Lifecycle Policies** to automatically purge untagged images older than 30 days to save cloud costs.

6.6 Vulnerability Scanning (DevSecOps)

- Integrate Aqua Trivy into the CI/CD pipeline to scan the container OS packages.
- Block the deployment instantly if Critical or High severity vulnerabilities are found, enforcing a strict zero-trust security gate.

6.7 Predictive Cost Analysis (FinOps)

- Generate a Terraform execution plan before any infrastructure is provisioned.
- Parse this plan through the Infracost API to fetch real-time AWS pricing and require explicit user approval for the estimated monthly run-rate.

6.8 Infrastructure Provisioning & Service Discovery

- Execute HashiCorp Terraform to construct the AWS networking topology (VPCs, public/private subnets, NAT Gateways).

- **Multi-Service Support:** Provision AWS Cloud Map for internal Service Discovery, allowing interconnected microservices to communicate securely over local DNS without hitting the public internet.

6.9 Distributed State Management

- Prevent the "Split-Brain" deployment collision problem when multiple developers deploy simultaneously.
- Implement remote state locking using Amazon S3 (with versioning) to store the terraform.tfstate files and Amazon DynamoDB to queue concurrent execution requests.

6.10 Persistent Storage & Backup Recovery (AWS EFS)

- Dynamically provision Amazon Elastic File System (EFS) for stateful workloads and persistent databases.
- Ensure data survival across container restarts by mounting EFS volumes with strict POSIX permissions, encrypted at rest via AWS KMS.

6.11 Zero-Downtime Container Orchestration & Event-Driven Rollbacks

- Deploy containers to serverless AWS ECS Fargate Spot instances with target-tracking CPU auto-scaling.
- **Spot Resilience:** Actively listen for AWS EventBridge 2-minute Spot Interruption notices to spin up replacement tasks instantly.
- **Layer 7 Health Checks:** Configure the ALB to constantly ping the /health endpoint. If a container fails, the system orchestrates a **Zero-Downtime Rollback** to the previously stable Git commit hash.

6.12 Dual-Stack Monitoring & Observability

- **Pipeline Stack:** Monitor GitHub Actions build times and Trivy scan durations.
- **Infrastructure Stack:** Stream (stdout) and (stderr) logs from ECS tasks to the frontend via Server-Sent Events (SSE). Integrate Prometheus to plot runtime telemetry (CPU/RAM usage, HTTP latency) on the dashboard.

6.13 AI-Powered Log Analysis & Chatbot

- Integrate an LLM-powered chatbot into the user dashboard.
- Automatically ingest logs from failed builds, failed Terraform applies, or Trivy security blocks.
- Use the LLM to parse these cryptic errors, pinpoint the exact point of failure, and provide the user with conversational, step-by-step remediation instructions.

6.14 Alerting & Notification System

- Track the lifecycle of every deployment (Queued, Building, Scanning, Provisioning, Deployed, Failed).

- Integrate **AWS Simple Notification Service (SNS)** to dispatch real-time webhooks to email for critical events.

6.15 Infrastructure Lifecycle & Demolition (Destroy)

- Provide a dual-confirmation workflow that executes terraform destroy to safely tear down active AWS environments.
- Explicitly skip the destruction of persistent EFS volumes to prevent accidental data loss.

6.16 Vendor Independence & Terraform Export (No Lock-in)

- Prevent platform lock-in by providing a "One-Click Export" feature.
- Allow users to download a .zip archive containing their generated Terraform scripts,giving them 100% ownership of their infrastructure as code.

6.17 Subscription & Billing Management

- **Tiered Access:** Implementation of "Free" and "Pro" plans to manage resource limits (e.g., number of active projects).
- **Payment Integration:** High-level integration with a standard payment gateway for secure checkout.
- **Billing Dashboard:** A simple interface for users to view their current plan, update payment methods, and download past invoices.
- **Usage Tracking:** Provide alerts if a user reaches their tier limit.

7. System Limitations/Constraints

- **Cloud Provider Lock-in:** The system's highly optimized Terraform modules are tailored specifically to AWS APIs. Multi-cloud deployment (Azure/GCP) is out of scope.
- **Spot Instance Volatility:** Because the platform relies on AWS Fargate Spot for extreme cost savings, compute nodes may occasionally be reclaimed by AWS during periods of regional capacity shortages.
- **Repository Platform:** The CI/CD pipeline currently requires the source code to be hosted on GitHub.
- **Template Mapping Boundaries:** The system will strictly deploy frameworks that match its pre-stored DevOps templates. Highly unconventional stacks will require manual Dockerfile inclusion by the user.

8. Data Gathering Approach

Data gathering for this complex orchestration system relies on a comprehensive study of cloud engineering documentation. We will dedicate substantial effort to comprehending AWS architectural best practices (ECS, Fargate, VPCs), HashiCorp Terraform documentation, and modern DevSecOps standards. Furthermore, we will continuously seek guidance from our supervisor, Sir Yaseen Mushtaq, to validate our infrastructure decisions and LLM prompt engineering for log analysis.

9. Tools and Technologies

Tool/Technology	Rationale
Next.js (React)	Frontend Development, UI/UX, & Chatbot Interface
NestJS & Node.js	Backend Orchestration Engine
PostgreSQL	Relational Database for User/Project Data
Redis (BullMQ)	Asynchronous Task Queuing for Deployments
AWS (ECS, ECR, S3, ALB)	Cloud Infrastructure Provider
HashiCorp Terraform	Infrastructure as Code (IaC) engine
Aqua Trivy & Infracost	Security Scanning & FinOps Cost Estimation
GitHub Actions	CI/CD Pipeline Automation
LLM API	AI-Powered Troubleshooting and Log Parsing

10. Project Stakeholders and Roles

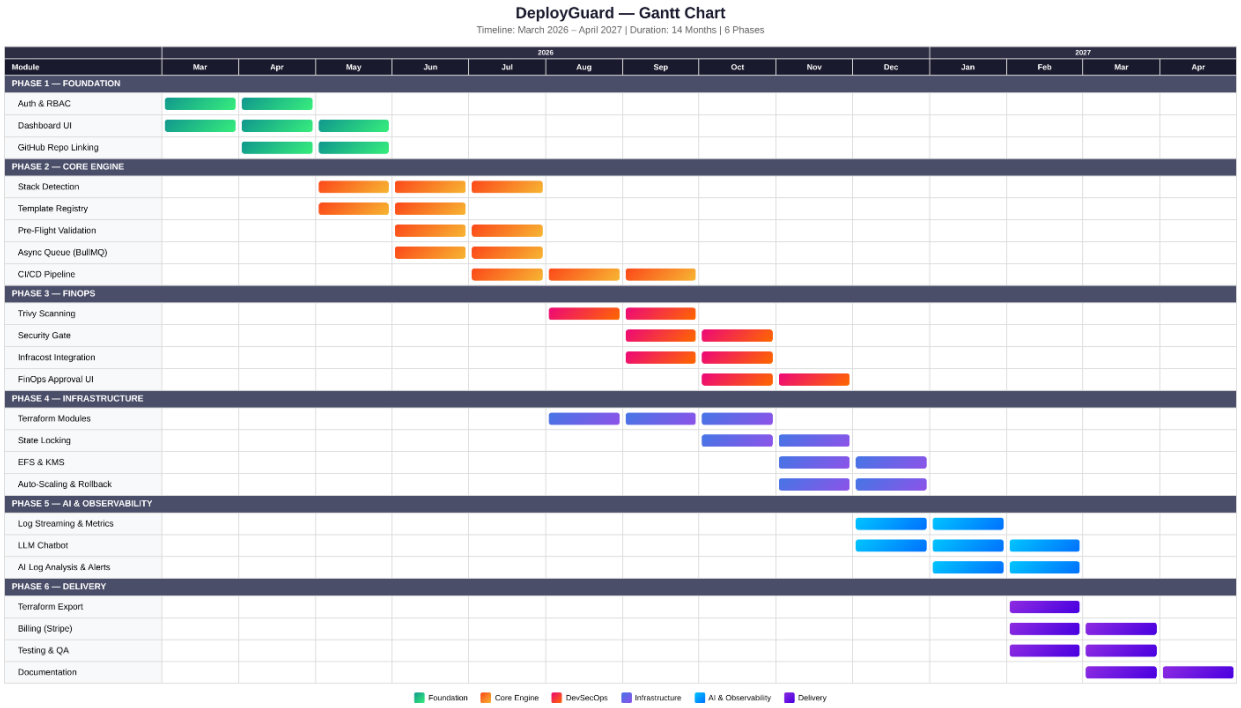
Stakeholder	Role
Air University Islamabad	Project Sponsor
Sir Yaseen Mushtaq	Project Supervisor
Final Year Project Committee	Evaluation and Quality Assurance of Project
Hassan Sajjad (232460)	Project Lead, Cloud Architect & IaC Engineer
Faria Fatima (233169)	Full-Stack Developer & Backend Orchestration Engineer
Tania Khawar (232378)	Full-Stack Developer & AI Integration Specialist

11. Module based Work Division

Student Name	Registration Number	Responsibility / Module / Feature
Hassan Sajjad	**232460**	**- Cloud Architect & Lead –**

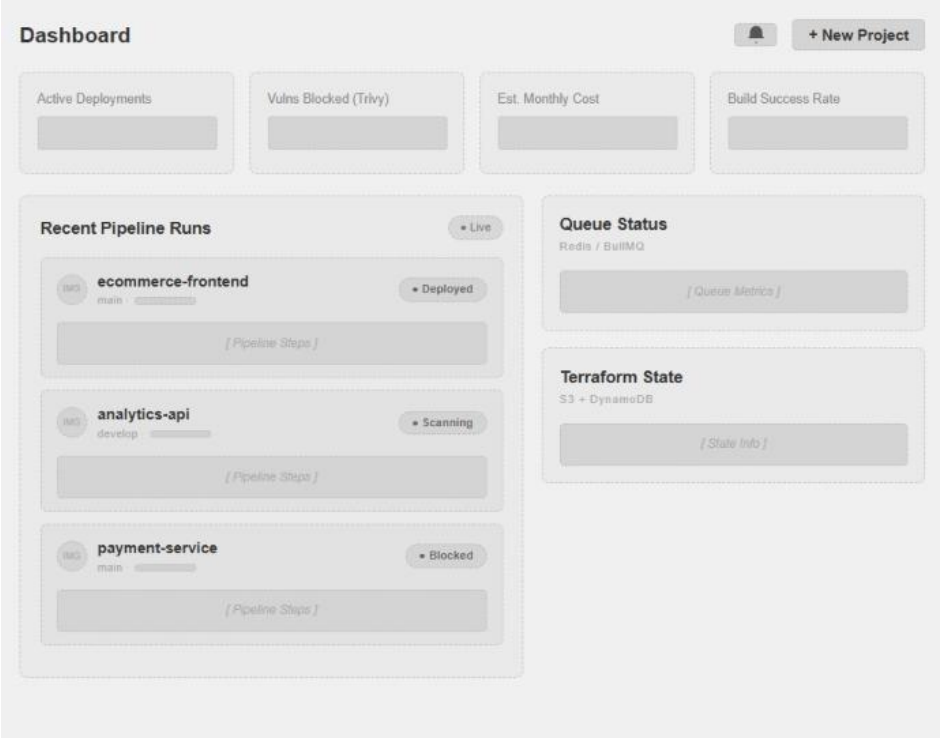
Student Name	Registration Number	Responsibility / Module / Feature
		(Responsible for the underlying cloud infrastructure, security gates, and lifecycle) * Infrastructure as Code (Terraform) * Distributed State Locking (S3/DynamoDB) * Security Scanning Gate (Aqua Trivy) * Predictive Cost Analysis (Infracost) * Lifecycle & Environment Demolition
Tania Khawar	**232378**	**- Frontend & AI Specialist -** (Responsible for user interfaces, AI troubleshooting, and deployment tracking) * Next.js User Dashboard & UI/UX * AI Log Analysis & Chatbot * Log Parsing & Remediation Logic (Optional) * CI/CD Pipeline Visualization * Real-time Monitoring & SSE Streaming
Faria Fatima	**233169**	**- Backend Orchestrator -** (Handles the central API logic, repository data, and background processing) * NestJS API & User Authentication * Targeted Repository Ingestion (Shallow Clones) * Asynchronous Queuing (Redis/BullMQ) * PostgreSQL Audit Logging * Payment & Subscription Integration

12. Gantt Chart



13. Mockups

Home/Dashboard Page



Description: This is the dashboard page, where users can view project summaries, system

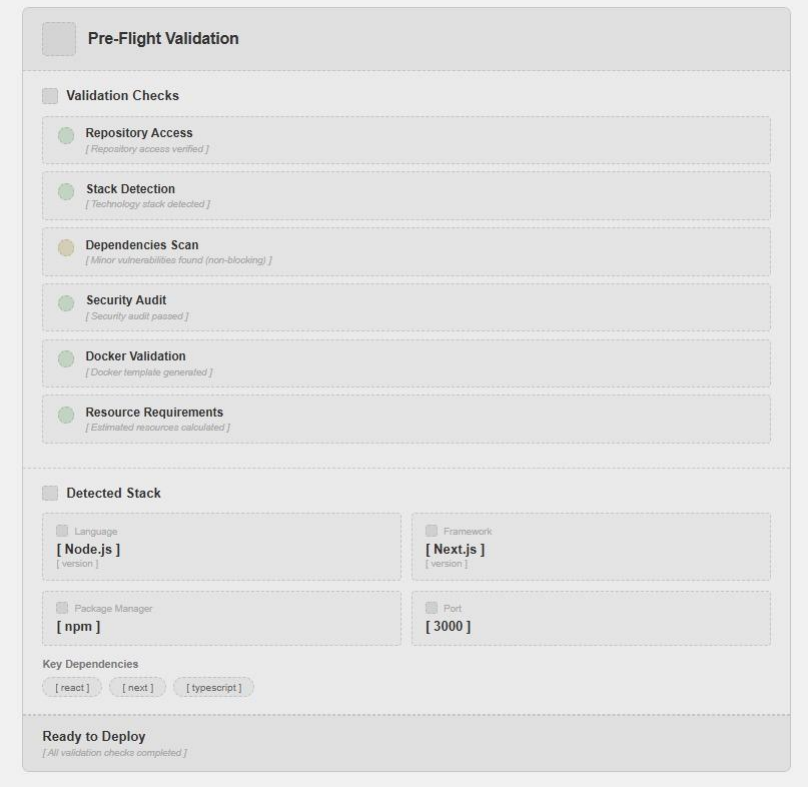
metrices, current status and recent notifications all in one place.

Project Creation/Work Space Page



Description: This is the Project Creation page where users link their GitHub repository, select the target branch, and define a project name before proceeding to the next configuration step.

Repository Stack Detection & Pre-Flight Page



Description: This is the stack detection and pre-flight page, where the system checks the project's technologies, validates the setup, and confirms everything is ready before deployment

Chatbot Page



Description: This is the AI troubleshooting page where users can analyze deployment logs and chat with a bot to resolve Terraform or build errors.

14. References

1. Amazon Web Services (AWS) Architecture Center. *Best Practices for Amazon ECS and AWS Fargate*.
2. HashiCorp Documentation. *Terraform State Management and Distributed Locking.*
3. Aqua Security. *Trivy: Comprehensive Security Scanner for Containers and Artifacts*.
4. Infracost Documentation. *Cloud Cost Estimation for Terraform*
5. Storment, J. R., & Fuller, M. (2020). *Cloud FinOps: Collaborative, Real-Time Cloud Financial Management.* O'Reilly Media.